BÖLÜM 16 / MODULE 16

İlişkisel Veritabanları: ACID, B-Tree İndeksleme & Sorgu Optimizasyonu

Relational Databases: ACID, B-Tree Indexes & Query Tuning

Transaction izolasyon seviyeleri (Dirty Read, Non-repeatable Read, Phantom Read), MVCC, B-Tree/B+Tree indeks anatomisi ve EXPLAIN analizi.

ACID transactional guarantees, ANSI SQL isolation anomalies, MVCC mechanics, B+Tree indexing internals, and query execution plans.

151

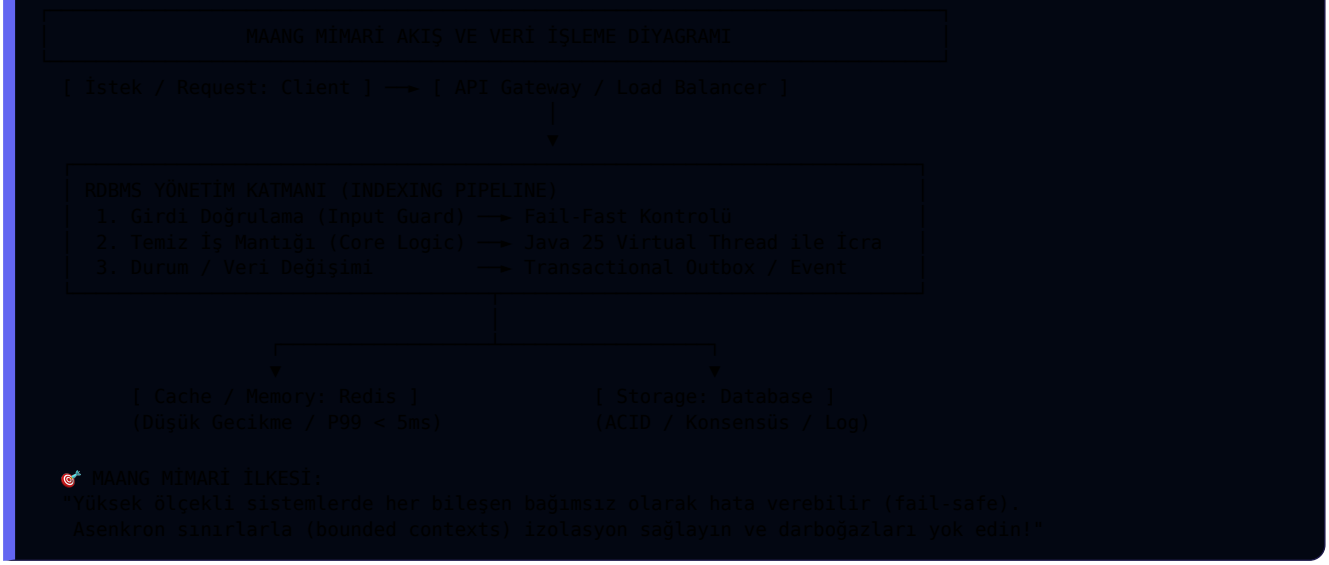
BÖLÜM: İLİŞKİSEL VERITABANLARI: ACID, B-TREE İNDEKSLEME & SORGU OPTIMIZASYONU | MODULE: RELATIONAL DATABASES: ACID, B-TREE INDEXES & QUERY TUNING

İlişkisel Veritabanları: ACID, B-Tree İndeksleme & Sorgu Optimizasyonu - Temel Kavramlar, Tarihsel Evrim ve Temel Mühendislik Çerçevesi

Relational Databases: ACID, B-Tree Indexes & Query Tuning - Core Concepts, Historical Evolution & Foundational Frameworks

⚡ Mimari Akış ve Süreç Şeması

⚡ Process & Architecture Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (RDBMS mikro-servisleri) ve otomatik yönlendirme robotları (Indexing akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

💡 ELI5 Real-Life Metaphor

🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (RDBMS microservices), and dynamic routing switches (Indexing pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde İlişkisel Veritabanları: ACID, B-Tree İndeksleme & Sorgu Optimizasyonu Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde RDBMS mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): Indexing katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

🔍 MAANG Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for Relational Databases: ACID, B-Tree Indexes & Query Tuning: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), RDBMS implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in Indexing don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```java
// Java 25 & Spring Boot: RDBMS & Indexing Kurumsal Uygulaması
package com.maang.se.module16.lesson01;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class RDBMSArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record RDBMSRequest(String transactionId, String payload, Instant timestamp) {
        public RDBMSRequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record RDBMSResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class RDBMSExecutionService {
        public RDBMSResult processRequest(RDBMSRequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new RDBMSResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering RDBMS Engine Başlatıldı...");
        var service = new RDBMSExecutionService();
        var req = new RDBMSRequest("TXN-" + System.currentTimeMillis(), "Valid Data", Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

RDBMS kurgulanırken Indexing bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling RDBMS logic directly to synchronous blocking calls into Indexing is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

🎯 Mülakatta Hatırlanacak Altın Kurallar

• RDBMS mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

🎯 Key Architectural Takeaways

• Design RDBMS components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

152

BÖLÜM: İLİŞKİSEL VERITABANLARI: ACID, B-TREE İNDEKSLEME & SORGU OPTIMIZASYONU | MODULE: RELATIONAL DATABASES: ACID, B-TREE INDEXES & QUERY TUNING

İlişkisel Veritabanları: ACID, B-Tree İndeksleme & Sorgu Optimizasyonu - Mimari Bileşenler, Çalışma Mekanizması ve Veri Akışı

Relational Databases: ACID, B-Tree Indexes & Query Tuning - Architectural Components, Internal Mechanics & Data Flow

⚡ Mimari Akış ve Süreç Şeması

⚡ Process & Architecture Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Real-Life Metaphor

🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (RDBMS mikro-servisleri) ve otomatik yönlendirme robotları (Indexing akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (RDBMS microservices), and dynamic routing switches (Indexing pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde İlişkisel Veritabanları: ACID, B-Tree İndeksleme & Sorgu Optimizasyonu Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde RDBMS mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): Indexing katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

🔍 MAANG Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for Relational Databases: ACID, B-Tree Indexes & Query Tuning: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), RDBMS implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in Indexing don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```java
// Java 25 & Spring Boot: RDBMS & Indexing Kurumsal Uygulaması
package com.maang.se.module16.lesson02;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class RDBMSArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record RDBMSRequest(String transactionId, String payload, Instant timestamp) {
        public RDBMSRequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record RDBMSResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class RDBMSExecutionService {
        public RDBMSResult processRequest(RDBMSRequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new RDBMSResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering RDBMS Engine Başlatıldı...");
        var service = new RDBMSExecutionService();
        var req = new RDBMSRequest("TXN-" + System.currentTimeMillis(), "Valid Data", Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

RDBMS kurgulanırken Indexing bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling RDBMS logic directly to synchronous blocking calls into Indexing is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

🎯 Mülakatta Hatırlanacak Altın Kurallar

• RDBMS mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

🎯 Key Architectural Takeaways

• Design RDBMS components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

153

BÖLÜM: İLİŞKİSEL VERITABANLARI: ACID, B-TREE İNDEKSLEME & SORGU OPTIMIZASYONU | MODULE: RELATIONAL DATABASES: ACID, B-TREE INDEXES & QUERY TUNING

İlişkisel Veritabanları: ACID, B-Tree İndeksleme & Sorgu Optimizasyonu - Tasarım İlkeleri, Sözleşmeler ve Temiz Arayüz Tanımları

Relational Databases: ACID, B-Tree Indexes & Query Tuning - Design Invariants, Contracts & Clean Interface Abstractions

⚡ Mimari Akış ve Süreç Şeması

⚡ Process & Architecture Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Real-Life Metaphor

🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (RDBMS mikro-servisleri) ve otomatik yönlendirme robotları (Indexing akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (RDBMS microservices), and dynamic routing switches (Indexing pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde İlişkisel Veritabanları: ACID, B-Tree İndeksleme & Sorgu Optimizasyonu Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde RDBMS mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): Indexing katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

🔍 MAANG Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for Relational Databases: ACID, B-Tree Indexes & Query Tuning: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), RDBMS implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in Indexing don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```java
// Java 25 & Spring Boot: RDBMS & Indexing Kurumsal Uygulaması
package com.maang.se.module16.lesson03;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class RDBMSArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record RDBMSRequest(String transactionId, String payload, Instant timestamp) {
        public RDBMSRequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record RDBMSResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class RDBMSExecutionService {
        public RDBMSResult processRequest(RDBMSRequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new RDBMSResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering RDBMS Engine Başlatıldı...");
        var service = new RDBMSExecutionService();
        var req = new RDBMSRequest("TXN-" + System.currentTimeMillis(), "Valid Data", Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

RDBMS kurgulanırken Indexing bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling RDBMS logic directly to synchronous blocking calls into Indexing is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

🎯 Mülakatta Hatırlanacak Altın Kurallar

• RDBMS mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

🎯 Key Architectural Takeaways

• Design RDBMS components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

154

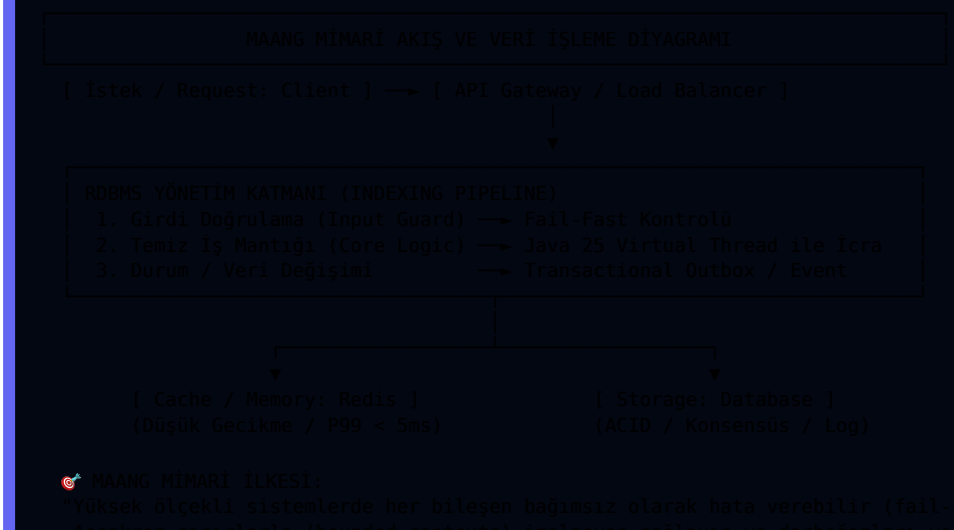
BÖLÜM: İLİŞKİSEL VERITABANLARI: ACID, B-TREE İNDEKSLEME & SORGU OPTIMIZASYONU | MODULE: RELATIONAL DATABASES: ACID, B-TREE INDEXES & QUERY TUNING

İlişkisel Veritabanları: ACID, B-Tree İndeksleme & Sorgu Optimizasyonu - Yaygın Anti-Pattern'ler, Tasarım Hataları ve Önleme Stratejileri

Relational Databases: ACID, B-Tree Indexes & Query Tuning - Common Anti-Patterns, Design Pitfalls & Prevention Strategies

⚡ Mimari Akış ve Süreç Şeması

⚡ Process & Architecture Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Real-Life Metaphor

🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (RDBMS mikro-servisleri) ve otomatik yönlendirme robotları (Indexing akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (RDBMS microservices), and dynamic routing switches (Indexing pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde İlişkisel Veritabanları: ACID, B-Tree İndeksleme & Sorgu Optimizasyonu Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde RDBMS mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): Indexing katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

🔍 MAANG Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for Relational Databases: ACID, B-Tree Indexes & Query Tuning: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), RDBMS implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in Indexing don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```java
// Java 25 & Spring Boot: RDBMS & Indexing Kurumsal Uygulaması
package com.maang.se.module16.lesson04;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class RDBMSArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record RDBMSRequest(String transactionId, String payload, Instant timestamp) {
        public RDBMSRequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record RDBMSResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class RDBMSExecutionService {
        public RDBMSResult processRequest(RDBMSRequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new RDBMSResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering RDBMS Engine Başlatıldı...");
        var service = new RDBMSExecutionService();
        var req = new RDBMSRequest("TXN-" + System.currentTimeMillis(), "Valid Data", Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

RDBMS kurgulanırken Indexing bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling RDBMS logic directly to synchronous blocking calls into Indexing is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

🎯 Mülakatta Hatırlanacak Altın Kurallar

• RDBMS mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

🎯 Key Architectural Takeaways

• Design RDBMS components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

155

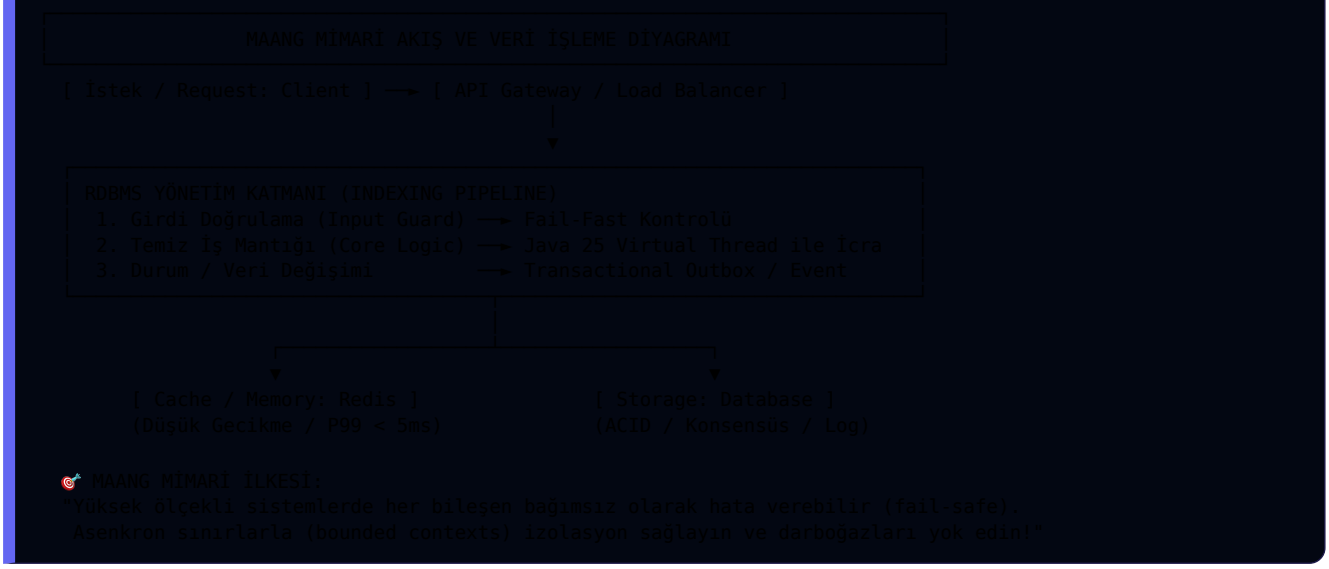
BÖLÜM: İLİŞKISEL VERITABANLARI: ACID, B-TREE İNDEKSLEME & SORGU OPTIMIZASYONU | MODULE: RELATIONAL DATABASES: ACID, B-TREE INDEXES & QUERY TUNING

İlişkisel Veritabanları: ACID, B-Tree İndeksleme & Sorgu Optimizasyonu - Yüksek Performans, Optimizasyon ve Bellek / CPU Yönetimi

Relational Databases: ACID, B-Tree Indexes & Query Tuning - High Performance, Optimization & Memory/CPU Resource Management

⚡ Mimari Akış ve Süreç Şeması

⚡ Process & Architecture Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Real-Life Metaphor

🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (RDBMS mikro-servisleri) ve otomatik yönlendirme robotları (Indexing akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (RDBMS microservices), and dynamic routing switches (Indexing pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde İlişkisel Veritabanları: ACID, B-Tree İndeksleme & Sorgu Optimizasyonu Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde RDBMS mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): Indexing katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

🔍 MAANG Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for Relational Databases: ACID, B-Tree Indexes & Query Tuning: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), RDBMS implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in Indexing don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```java
// Java 25 & Spring Boot: RDBMS & Indexing Kurumsal Uygulaması
package com.maang.se.module16.lesson05;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class RDBMSArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record RDBMSRequest(String transactionId, String payload, Instant timestamp) {
        public RDBMSRequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record RDBMSResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class RDBMSExecutionService {
        public RDBMSResult processRequest(RDBMSRequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new RDBMSResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering RDBMS Engine Başlatıldı...");
        var service = new RDBMSExecutionService();
        var req = new RDBMSRequest("TXN-" + System.currentTimeMillis(), "Valid Data", Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

RDBMS kurgulanırken Indexing bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling RDBMS logic directly to synchronous blocking calls into Indexing is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

🎯 Mülakatta Hatırlanacak Altın Kurallar

• RDBMS mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

🎯 Key Architectural Takeaways

• Design RDBMS components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

156

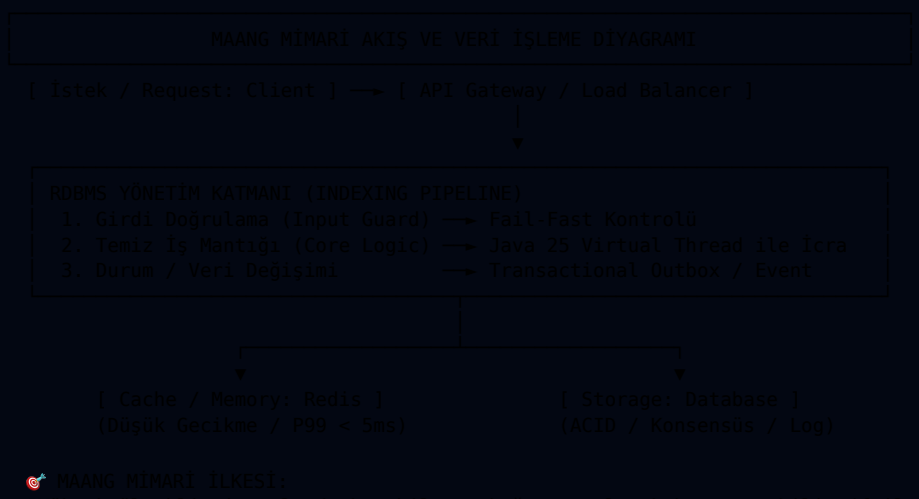
BÖLÜM: İLİŞKİSEL VERITABANLARI: ACID, B-TREE İNDEKSLEME & SORGU OPTIMIZASYONU | MODULE: RELATIONAL DATABASES: ACID, B-TREE INDEXES & QUERY TUNING

İlişkisel Veritabanları: ACID, B-Tree İndeksleme & Sorgu Optimizasyonu - Eşzamanlılık, Thread Güvenliği ve Dağıtık Sistem Etkileri

Relational Databases: ACID, B-Tree Indexes & Query Tuning - Concurrency, Thread-Safety & Distributed Systems Implications

⚡ Mimari Akış ve Süreç Şeması

⚡ Process & Architecture Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (RDBMS mikro-servisleri) ve otomatik yönlendirme robotları (Indexing akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

💡 ELI5 Real-Life Metaphor

🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (RDBMS microservices), and dynamic routing switches (Indexing pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde İlişkisel Veritabanları: ACID, B-Tree İndeksleme & Sorgu Optimizasyonu Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde RDBMS mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): Indexing katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

🔍 MAANG Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for Relational Databases: ACID, B-Tree Indexes & Query Tuning: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), RDBMS implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in Indexing don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```java
// Java 25 & Spring Boot: RDBMS & Indexing Kurumsal Uygulaması
package com.maang.se.module16.lesson06;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class RDBMSArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record RDBMSRequest(String transactionId, String payload, Instant timestamp) {
        public RDBMSRequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record RDBMSResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class RDBMSExecutionService {
        public RDBMSResult processRequest(RDBMSRequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new RDBMSResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering RDBMS Engine Başlatıldı...");
        var service = new RDBMSExecutionService();
        var req = new RDBMSRequest("TXN-" + System.currentTimeMillis(), "Valid Data", Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

RDBMS kurgulanırken Indexing bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling RDBMS logic directly to synchronous blocking calls into Indexing is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

🎯 Mülakatta Hatırlanacak Altın Kurallar

• RDBMS mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

🎯 Key Architectural Takeaways

• Design RDBMS components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

157

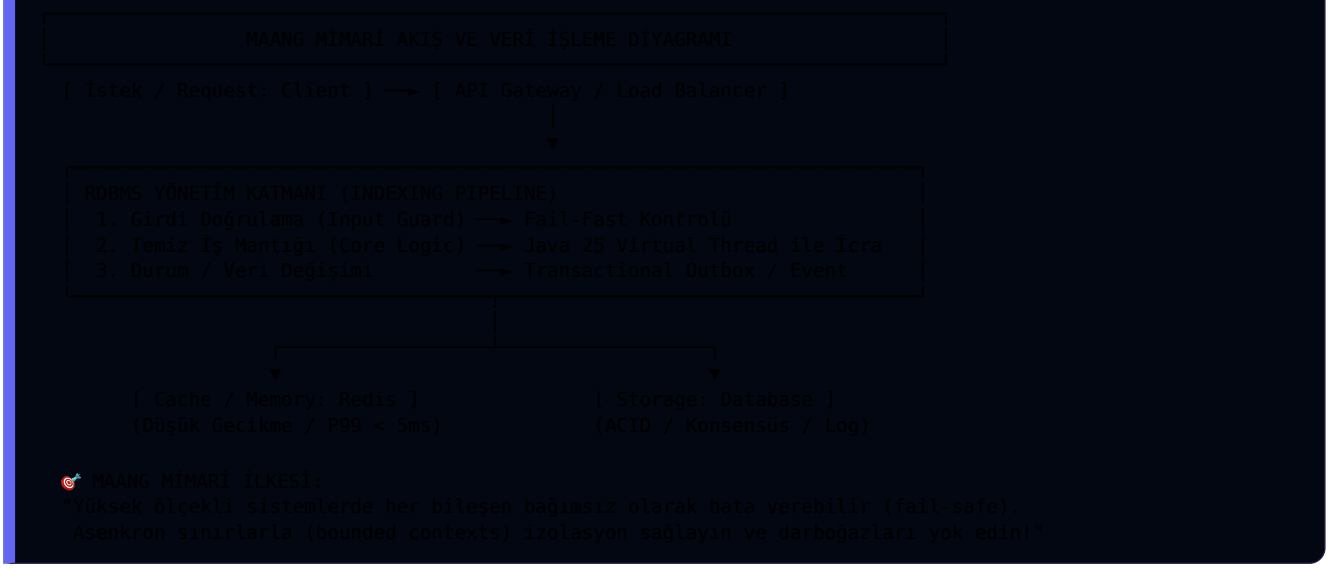
BÖLÜM: İLİŞKİSEL VERITABANLARI: ACID, B-TREE İNDEKSLEME & SORGU OPTIMIZASYONU | MODULE: RELATIONAL DATABASES: ACID, B-TREE INDEXES & QUERY TUNING

İlişkisel Veritabanları: ACID, B-Tree İndeksleme & Sorgu Optimizasyonu - Hata Yönetimi, Dayanıklılık (Resilience) ve Geri Düşme (Fallback)

Relational Databases: ACID, B-Tree Indexes & Query Tuning - Fault Handling, Resilience Patterns & Graceful Fallback Strategies

⚡ Mimari Akış ve Süreç Şeması

⚡ Process & Architecture Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (RDBMS mikro-servisleri) ve otomatik yönlendirme robotları (Indexing akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

💡 ELI5 Real-Life Metaphor

🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (RDBMS microservices), and dynamic routing switches (Indexing pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde İlişkisel Veritabanları: ACID, B-Tree İndeksleme & Sorgu Optimizasyonu Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde RDBMS mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): Indexing katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

🔍 MAANG Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for Relational Databases: ACID, B-Tree Indexes & Query Tuning: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), RDBMS implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in Indexing don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```java
// Java 25 & Spring Boot: RDBMS & Indexing Kurumsal Uygulaması
package com.maang.se.module16.lesson07;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class RDBMSArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record RDBMSRequest(String transactionId, String payload, Instant timestamp) {
        public RDBMSRequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record RDBMSResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class RDBMSExecutionService {
        public RDBMSResult processRequest(RDBMSRequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new RDBMSResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering RDBMS Engine Başlatıldı...");
        var service = new RDBMSExecutionService();
        var req = new RDBMSRequest("TXN-" + System.currentTimeMillis(), "Valid Data", Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

RDBMS kurgulanırken Indexing bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling RDBMS logic directly to synchronous blocking calls into Indexing is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

🎯 Mülakatta Hatırlanacak Altın Kurallar

• RDBMS mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

🎯 Key Architectural Takeaways

• Design RDBMS components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

158

BÖLÜM: İLİŞKİSEL VERITABANLARI: ACID, B-TREE İNDEKSLEME & SORGU OPTIMIZASYONU | MODULE: RELATIONAL DATABASES: ACID, B-TREE INDEXES & QUERY TUNING

İlişkisel Veritabanları: ACID, B-Tree İndeksleme & Sorgu Optimizasyonu - Test Stratejileri, Test Edilebilirlik ve Doğrulama Hatları

Relational Databases: ACID, B-Tree Indexes & Query Tuning - Testing Strategies, Testability & Automated Verification Pipelines

⚡ Mimari Akış ve Süreç Şeması

⚡ Process & Architecture Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Real-Life Metaphor

🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (RDBMS mikro-servisleri) ve otomatik yönlendirme robotları (Indexing akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (RDBMS microservices), and dynamic routing switches (Indexing pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde İlişkisel Veritabanları: ACID, B-Tree İndeksleme & Sorgu Optimizasyonu Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde RDBMS mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): Indexing katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

🔍 MAANG Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for Relational Databases: ACID, B-Tree Indexes & Query Tuning: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), RDBMS implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in Indexing don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```java
// Java 25 & Spring Boot: RDBMS & Indexing Kurumsal Uygulaması
package com.maang.se.module16.lesson08;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class RDBMSArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record RDBMSRequest(String transactionId, String payload, Instant timestamp) {
        public RDBMSRequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record RDBMSResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class RDBMSExecutionService {
        public RDBMSResult processRequest(RDBMSRequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new RDBMSResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering RDBMS Engine Başlatıldı...");
        var service = new RDBMSExecutionService();
        var req = new RDBMSRequest("TXN-" + System.currentTimeMillis(), "Valid Data", Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

RDBMS kurgulanırken Indexing bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling RDBMS logic directly to synchronous blocking calls into Indexing is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

🎯 Mülakatta Hatırlanacak Altın Kurallar

• RDBMS mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

🎯 Key Architectural Takeaways

• Design RDBMS components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

159

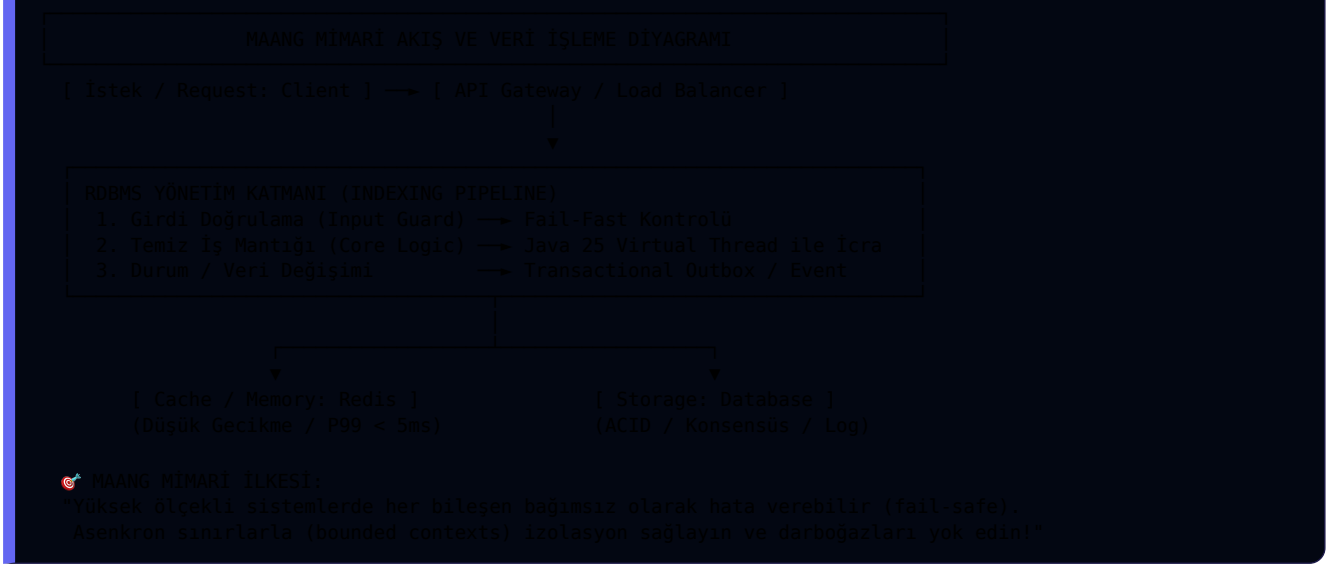
BÖLÜM: İLİŞKİSEL VERITABANLARI: ACID, B-TREE İNDEKSLEME & SORGU OPTIMIZASYONU | MODULE: RELATIONAL DATABASES: ACID, B-TREE INDEXES & QUERY TUNING

İlişkisel Veritabanları: ACID, B-Tree İndeksleme & Sorgu Optimizasyonu - Gözlemlenebilirlik, Metrikler, İzleme ve Canlı Ortam Hata Ayıklama

Relational Databases: ACID, B-Tree Indexes & Query Tuning - Observability, Telemetry, Tracing & Production Troubleshooting

⚡ Mimari Akış ve Süreç Şeması

⚡ Process & Architecture Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Real-Life Metaphor

🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (RDBMS mikro-servisleri) ve otomatik yönlendirme robotları (Indexing akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (RDBMS microservices), and dynamic routing switches (Indexing pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde İlişkisel Veritabanları: ACID, B-Tree İndeksleme & Sorgu Optimizasyonu Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde RDBMS mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): Indexing katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

🔍 MAANG Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for Relational Databases: ACID, B-Tree Indexes & Query Tuning: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), RDBMS implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in Indexing don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```java
// Java 25 & Spring Boot: RDBMS & Indexing Kurumsal Uygulaması
package com.maang.se.module16.lesson09;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class RDBMSArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record RDBMSRequest(String transactionId, String payload, Instant timestamp) {
        public RDBMSRequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record RDBMSResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class RDBMSExecutionService {
        public RDBMSResult processRequest(RDBMSRequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new RDBMSResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering RDBMS Engine Başlatıldı...");
        var service = new RDBMSExecutionService();
        var req = new RDBMSRequest("TXN-" + System.currentTimeMillis(), "Valid Data", Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

RDBMS kurgulanırken Indexing bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling RDBMS logic directly to synchronous blocking calls into Indexing is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

🎯 Mülakatta Hatırlanacak Altın Kurallar

• RDBMS mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

🎯 Key Architectural Takeaways

• Design RDBMS components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

160

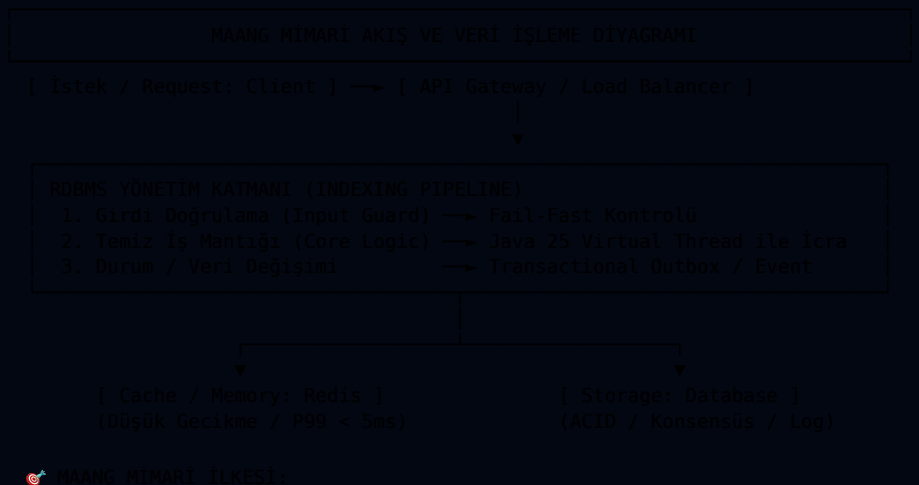
BÖLÜM: İLİŞKİSEL VERITABANLARI: ACID, B-TREE İNDEKSLEME & SORGU OPTIMIZASYONU | MODULE: RELATIONAL DATABASES: ACID, B-TREE INDEXES & QUERY TUNING

İlişkisel Veritabanları: ACID, B-Tree İndeksleme & Sorgu Optimizasyonu - MAANG Mülakat Ustalığı, Mimari Ödünleşimler (Trade-Offs) & Staff+ Vizyonu

Relational Databases: ACID, B-Tree Indexes & Query Tuning - MAANG Interview Mastery, Architectural Trade-Offs & Staff+ Vision

⚡ Mimari Akış ve Süreç Şeması

⚡ Process & Architecture Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Real-Life Metaphor

🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (RDBMS mikro-servisleri) ve otomatik yönlendirme robotları (Indexing akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (RDBMS microservices), and dynamic routing switches (Indexing pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde İlişkisel Veritabanları: ACID, B-Tree İndeksleme & Sorgu Optimizasyonu Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde RDBMS mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): Indexing katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

🔍 MAANG Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for Relational Databases: ACID, B-Tree Indexes & Query Tuning: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), RDBMS implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in Indexing don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```java
// Java 25 & Spring Boot: RDBMS & Indexing Kurumsal Uygulaması
package com.maang.se.module16.lesson10;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class RDBMSArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record RDBMSRequest(String transactionId, String payload, Instant timestamp) {
        public RDBMSRequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record RDBMSResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class RDBMSExecutionService {
        public RDBMSResult processRequest(RDBMSRequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new RDBMSResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering RDBMS Engine Başlatıldı...");
        var service = new RDBMSExecutionService();
        var req = new RDBMSRequest("TXN-" + System.currentTimeMillis(), "Valid Data", Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

RDBMS kurgulanırken Indexing bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling RDBMS logic directly to synchronous blocking calls into Indexing is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

🎯 Mülakatta Hatırlanacak Altın Kurallar

• RDBMS mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

🎯 Key Architectural Takeaways

• Design RDBMS components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.